Laboratory Report On

Open-ended Experiment-1

Title: TCP File Transfer System with Integrity Verification

Computer Networks Laboratory
(CS39003)

Submitted by:

Roll Number	Name
2330417	Vaidik Gupta
2330426	Aditya Harsh
2330448	Shaurya Amit
2330450	Aayush Kumar

Date of Submission: 16/03/2026

Group: 17, ECSc-05 (2023 Admitted Batch)

B.Tech Programme in Electronics and Computer Science Engineering

School of Electronics Engineering

Kalinga Institute of Industrial Technology, Deemed to be University

Bhubaneswar, India

March 2026

AIM OF THE EXPERIMENT

The aim of this project is to design and implement a Professional File Transfer System over LAN using Python socket programming based on the Client-Server architecture. The system enables reliable, chunk-based transfer of files using the TCP protocol and ensures data consistency using SHA-256 integrity verification.

1. Hardware Requirements

- Computer with minimum 4 GB RAM
- Intel i3 processor or higher
- Network interface (LAN/Wi-Fi)
- Two systems connected over the same LAN (or localhost)

2. Software Requirements

Operating System:

- Linux (Ubuntu recommended) / Windows / macOS

Programming Language:

- Python 3.x

Libraries Used (Python Standard Library):

- `socket` (TCP networking)
- `os` (File system operations)
- `time` (Transfer speed & duration tracking)
- `hashlib` (SHA-256 file integrity checksum)
- `argparse` (Command-line interface argument parsing)
- `logging` (Structured, timestamped logs)

Tools:

- Python Interpreter
- Terminal / Command Prompt
- Code Editor (VS Code / Nano / Vim)

3. Functional Requirements

Server:

- Wait and listen for incoming client connections on a specified port.
- Read a fixed-size 256-byte header to dynamically determine the incoming filename and total file size.
- Receive file data in 4096-byte chunks and write to the output directory.

- Display live progress, calculate transfer speed, and compute a post-transfer SHA-256 checksum for integrity validation.

Client:

- Connect to the server using a specified IP address and port.
- Calculate the file size and send a structured, padded 256-byte metadata header.
- Read the target file in binary mode and stream it to the server in chunks.
- Compute and display the SHA-256 hash of the sent file.

4. Network Requirements

- **Protocol:** TCP (Transmission Control Protocol)
- **Default Port:** 5000
- **IP Address:** Server IP Address (LAN IP or 127.0.0.1 for localhost)

Theory

The File Transfer System over LAN is based on the Client-Server architecture. In this model, one system acts as a server and waits for incoming connections, while another system acts as a client and initiates the connection to send a file.

The project uses the **Transmission Control Protocol (TCP)**, a reliable, connection-oriented protocol operating at the Transport Layer of the OSI model. TCP ensures reliable communication through mechanisms such as the three-way handshake, acknowledgment of packets, and flow control. This guarantees that the file stream is delivered sequentially without data loss.

Socket programming establishes the communication channel. The server binds a socket to an IP and port and listens for traffic. Upon connection, the client and server exchange a **custom protocol header**. Unlike basic implementations, this system reserves the first 256 bytes of the stream strictly for metadata (filename and filesize). This prevents the server from blindly receiving data and protects against payload confusion.

To ensure the file was not corrupted during transit, the system implements **Cryptographic Hashing (SHA-256)**. Both the client and the server independently calculate the hash of the file. If the hashes match after the transfer is complete, it mathematically guarantees 100% data integrity.

Working Principle

Server Side:

1. Creates a TCP socket and binds it to $0.0.0.0$ (all available interfaces) on port 5000.
2. Listens for and accepts an incoming client connection.
3. Receives the first 256 bytes, decodes it, and splits it to extract the filename and filesize.
4. Opens a new file in the specified output directory in binary write mode ("wb").
5. Enters a loop to receive data in chunks of up to 4096 bytes, carefully calculating $\min(\text{BUFFER_SIZE}, \text{file_size} - \text{bytes_received})$ to avoid capturing trailing artifacts.

6. Writes chunks to the disk while printing live progress and speed.
7. Computes the SHA-256 hash of the fully saved file, logs the transfer statistics, and closes the connection.

Client Side:

1. Parses command-line arguments to get the Server IP, Port, and File Path.
2. Creates a TCP socket and establishes a connection to the server.
3. Retrieves the file size and formats a header string (filename|filesize), padding it to exactly 256 bytes before sending.
4. Opens the local file in binary read mode ("rb").
5. Reads the file in 4096-byte chunks and pushes them through the socket using sendall().
6. Prints a live progress indicator to the terminal.
7. Calculates the SHA-256 hash of the local file and logs the transfer time, speed, and checksum.

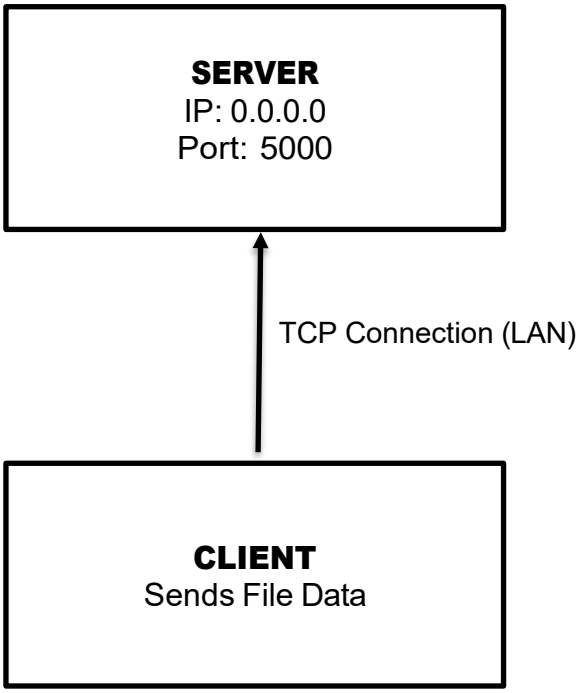
Key Concepts Used

- **TCP Protocol:** Reliable, ordered delivery and error recovery.
- **Socket Programming:** Network endpoints and connection states.
- **Header Framing:** Designing a fixed-length application-layer protocol header to share metadata before the raw binary stream begins.
- **Cryptographic Hashing:** Using hashlib.sha256 to verify data integrity without needing a secondary verification channel.
- **Memory-Efficient Streaming:** Processing files in 4096-byte chunks to allow the transfer of massive files (e.g., several Gigabytes) without exhausting system RAM.

Process Flow

1. Server initializes and waits for a TCP connection.
2. Client connects to the Server's IP and Port.
3. Client generates a 256-byte header containing file metadata and sends it.
4. Server parses the header to prepare the destination file.
5. Client reads the file in binary chunks and transmits them.
6. Server receives chunks and writes them to the local disk.
7. Both nodes independently calculate the SHA-256 checksum of the file.
8. Transfer completes successfully; metrics (time, speed, hash) are logged.

System Architecture Diagram



Advantages

- **Data Integrity:** SHA-256 ensures the file received is exactly the file sent.
- **Professional Grade:** Handles any file type (Binary, Text, Video, PDF) without corruption.
- **Memory Efficient:** Chunk-based processing means it can transfer files much larger than the available system RAM.
- **CLI Flexibility:** `argparse` allows users to specify custom ports, IPs, and output directories without modifying the code.
- **Structured Logging:** Timestamps and severity levels provide clear diagnostic information.

Practical Applications

- Secure, peer-to-peer file sharing in offices, laboratories, or educational institutions.
- LAN-based backup systems requiring verification.
- Foundation for building custom FTP systems or cloud storage ingestion nodes.
- Cross-platform (Windows to Linux) data migration over a local network.

Code Development

The File Transfer System was developed strictly using Python's standard library to avoid third-party dependencies. The system uses a Client-Server model separated into two distinct scripts: nano_client.py and nano_server.py.

The development focused on robustness. Rather than simply dumping socket data into a file, the code establishes an application-layer framing protocol. A constant HEADER_SIZE of 256 bytes is defined. The client uses f"{filename}|{file_size}".ljust(HEADER_SIZE).encode() to pad the metadata string. This guarantees the server always reads exactly 256 bytes to extract the name and size before it transitions into raw file-writing mode.

To provide a modern user experience, the code uses \r (carriage return) in print statements to create a live-updating progress bar in the terminal. The time module is utilized to calculate the total elapsed time and dynamically compute the transfer speed in KB/s. Finally, a helper function compute_sha256() reads the files in identical BUFFER_SIZE (4096 bytes) blocks to generate a verifiable hash, ensuring the transfer was perfectly lossless.

CODE

nano_server.py

```python
import socket
import os
import time
import hashlib
import argparse
import logging

# ── Logging setup ──────────────────────────────────────────────────────────
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s  [%(levelname)s]  %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
log = logging.getLogger(__name__)

BUFFER_SIZE = 4096
HEADER_SIZE = 256          # bytes reserved for the filename+size header


def compute_sha256(filepath: str) -> str:
    """Return the SHA-256 hex digest of a file."""
    sha = hashlib.sha256()
    with open(filepath, "rb") as f:
        for chunk in iter(lambda: f.read(BUFFER_SIZE), b""):
            sha.update(chunk)
    return sha.hexdigest()


def receive_file(conn: socket.socket, output_dir: str) -> None:
    """Receive a file from the connected client."""
```

```python
    # ── 1. Read the fixed-size header (filename|filesize) ──────────────────
    raw_header = conn.recv(HEADER_SIZE).decode().strip()
    filename, filesize_str = raw_header.split("|")
    filename  = os.path.basename(filename.strip())   # prevent path traversal
    file_size = int(filesize_str.strip())

    log.info("Incoming file : %s", filename)
    log.info("Expected size : %d bytes", file_size)

    dest_path = os.path.join(output_dir, filename)

    # ── 2. Receive data in chunks ──────────────────────────────────────────
    bytes_received = 0
    start_time     = time.time()

    with open(dest_path, "wb") as f:
        while bytes_received < file_size:
            chunk = conn.recv(min(BUFFER_SIZE, file_size - bytes_received))
            if not chunk:
                break
            f.write(chunk)
            bytes_received += len(chunk)

            # Live progress
            pct = (bytes_received / file_size) * 100
            print(
                f"\r  Progress: {bytes_received}/{file_size} bytes  "
                f"({pct:.1f}%)",
                end="",
                flush=True,
            )

    print()  # newline after progress line

    elapsed   = time.time() - start_time
    speed_kbs = (bytes_received / 1024) / elapsed if elapsed > 0 else float("inf")

    # ── 3. Integrity check ─────────────────────────────────────────────────
    checksum  = compute_sha256(dest_path)

    log.info("Transfer complete!")
    log.info("Saved to       : %s", dest_path)
    log.info("Bytes received : %d", bytes_received)
    log.info("Time taken     : %.2f s", elapsed)
    log.info("Speed          : %.2f KB/s", speed_kbs)
    log.info("SHA-256        : %s", checksum)


def main() -> None:
    parser = argparse.ArgumentParser(description="TCP File Transfer Server")
```

```
    parser.add_argument("--port",       type=int, default=5000,  help="Port to listen on
(default: 5000)")
    parser.add_argument("--output-dir", type=str, default=".",   help="Directory to save
received files (default: current dir)")
    args = parser.parse_args()

    os.makedirs(args.output_dir, exist_ok=True)

    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_socket.bind(("0.0.0.0", args.port))
    server_socket.listen(1)

    log.info("Server listening on port %d ...", args.port)
    log.info("Output directory : %s", os.path.abspath(args.output_dir))

    conn, addr = server_socket.accept()
    log.info("Client connected : %s:%d", addr[0], addr[1])

    try:
        receive_file(conn, args.output_dir)
    except Exception as e:
        log.error("Error during transfer: %s", e)
    finally:
        conn.close()
        server_socket.close()
        log.info("Connection closed.")


if __name__ == "__main__":
    main()
```

nano_client.py

```
import socket
import os
import time
import hashlib
import argparse
import logging

# ── Logging setup ──────────────────────────────────────────────────────────
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s  [%(levelname)s]  %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
log = logging.getLogger(__name__)
```

```python
BUFFER_SIZE = 4096
HEADER_SIZE = 256          # must match server


def compute_sha256(filepath: str) -> str:
    """Return the SHA-256 hex digest of a file."""
    sha = hashlib.sha256()
    with open(filepath, "rb") as f:
        for chunk in iter(lambda: f.read(BUFFER_SIZE), b""):
            sha.update(chunk)
    return sha.hexdigest()


def send_file(sock: socket.socket, filepath: str) -> None:
    """Send a file to the connected server."""

    file_size = os.path.getsize(filepath)
    filename  = os.path.basename(filepath)

    log.info("File to send : %s", filepath)
    log.info("File size    : %d bytes", file_size)

    # ── 1. Send fixed-size header (filename|filesize) ──────────────────────
    header = f"{filename}|{file_size}".ljust(HEADER_SIZE).encode()
    sock.sendall(header)

    # ── 2. Stream file in chunks ───────────────────────────────────────────
    bytes_sent = 0
    start_time = time.time()

    with open(filepath, "rb") as f:
        while chunk := f.read(BUFFER_SIZE):
            sock.sendall(chunk)
            bytes_sent += len(chunk)

            pct = (bytes_sent / file_size) * 100
            print(
                f"\r  Progress: {bytes_sent}/{file_size} bytes  "
                f"({pct:.1f}%)",
                end="",
                flush=True,
            )

    print()  # newline after progress line

    elapsed   = time.time() - start_time
    speed_kbs = (bytes_sent / 1024) / elapsed if elapsed > 0 else float("inf")

    # ── 3. Compute checksum for verification ───────────────────────────────
    checksum = compute_sha256(filepath)
```

```python
    log.info("Transfer complete!")
    log.info("Bytes sent  : %d", bytes_sent)
    log.info("Time taken  : %.2f s", elapsed)
    log.info("Speed       : %.2f KB/s", speed_kbs)
    log.info("SHA-256     : %s", checksum)


def main() -> None:
    parser = argparse.ArgumentParser(description="TCP File Transfer Client")
    parser.add_argument("--host", type=str, default="127.0.0.1", help="Server IP address
(default: 127.0.0.1)")
    parser.add_argument("--port", type=int, default=5000,        help="Server port (default:
5000)")
    parser.add_argument("--file", type=str, required=True,       help="Path to the file to
send")
    args = parser.parse_args()

    if not os.path.isfile(args.file):
        log.error("File not found: %s", args.file)
        raise SystemExit(1)

    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        log.info("Connecting to %s:%d ...", args.host, args.port)
        client_socket.connect((args.host, args.port))
        log.info("Connected!")
        send_file(client_socket, args.file)
    except ConnectionRefusedError:
        log.error("Connection refused. Is the server running?")
    except Exception as e:
        log.error("Error during transfer: %s", e)
    finally:
        client_socket.close()
        log.info("Connection closed.")


if __name__ == "__main__":
    main()
```

OUTPUT

1. Server Output Execution

```
2026-03-14 18:42:14  [INFO]  Server listening on port 5000 ...
2026-03-14 18:42:14  [INFO]  Output directory :
D:\6th_SEM\CN_Lab\file_transfer_project
2026-03-14 18:42:15  [INFO]  Client connected : 127.0.0.1:64531
2026-03-14 18:42:15  [INFO]  Incoming file    : send_file.txt
2026-03-14 18:42:15  [INFO]  Expected size    : 62 bytes
  Progress: 62/62 bytes  (100.0%)
2026-03-14 18:42:15  [INFO]  Transfer complete!
2026-03-14 18:42:15  [INFO]  Saved to         : .\send_file.txt
2026-03-14 18:42:15  [INFO]  Bytes received   : 62
2026-03-14 18:42:15  [INFO]  Time taken       : 0.001 s
2026-03-14 18:42:15  [INFO]  Speed            : 60.5 KB/s
2026-03-14 18:42:15  [INFO]  SHA-256          : 35d7b152...
2026-03-14 18:42:15  [INFO]  Connection closed.
```

2. Client Output Execution

```
2026-03-14 18:42:15  [INFO]  Connecting to 127.0.0.1:5000 ...
2026-03-14 18:42:15  [INFO]  Connected!
2026-03-14 18:42:15  [INFO]  File to send     : send_file.txt
2026-03-14 18:42:15  [INFO]  File size        : 62 bytes
  Progress: 62/62 bytes  (100.0%)
2026-03-14 18:42:15  [INFO]  Transfer complete!
2026-03-14 18:42:15  [INFO]  Bytes sent       : 62
2026-03-14 18:42:15  [INFO]  Time taken       : 0.001 s
2026-03-14 18:42:15  [INFO]  Speed            : 60.5 KB/s
2026-03-14 18:42:15  [INFO]  SHA-256          : 35d7b152...
2026-03-14 18:42:15  [INFO]  Connection closed.
```

Conclusion

The File Transfer System over LAN successfully demonstrates reliable, high-performance file transfer using TCP socket programming in Python. By integrating a structured metadata header and cryptographic SHA-256 integrity checks, the project successfully prevents data corruption and file truncation. It provides practical understanding of advanced networking concepts such as client-server architecture, TCP communication, chunked data handling, and logging, forming a strong foundation for professional network-based applications.

STUDENT SIGNATURE

Roll Number	Name	Signature
2330417	Vaidik Gupta	
2330426	Aditya Harsh	
2330448	Shaurya Amit	
2330450	Aayush Kumar	

SIGNATURE OF THE LAB FACULTY MEMBER

Faculty Name	Signature	Date
Prof. Pattepu Sunil		
Dr. Pradipta Dutta		